Reproducible Data Science Projects

A Practical Guide

Table of contents

1 Introduction **2**
- 1.1 Two complementary dimensions of reproducibility . . . 2
- 1.2 Relationship to CRISP-DM . . . 3

2 Organization of a Reproducible Research Project **3**
- 2.1 The repository as the unit of reproducibility . . . 3
- 2.2 A minimal directory structure . . . 4
- 2.3 Source files and generated artifacts . . . 5
- 2.4 Naming and paths . . . 5

3 Computational Notebooks **5**
- 3.1 The role of Quarto notebooks . . . 5
- 3.2 One analytical responsibility per notebook . . . 6
- 3.3 A standard notebook anatomy . . . 6
- 3.4 Independent execution and explicit dependencies . . . 7
- 3.5 Computational transparency and reproducibility . . . 7

4 Scientific Reports **7**
- 4.1 Reports as scientific communication . . . 7
- 4.2 Reports and notebooks as complementary records . . . 8
- 4.3 Relationship to reporting milestones . . . 8

5 Modular and Automated Workflows **8**
- 5.1 Analytical versus software modularity . . . 8
- 5.2 Reusable R functions and `src/` . . . 9
- 5.3 Executable scripts and `scripts/` . . . 9
- 5.4 From interactive analysis to an automated pipeline . . . 9
- 5.5 Reproducibility checks . . . 10

6 Concluding Principles 10

1 Introduction

A Data Science research project involves more than applying statistical methods to a dataset. It requires formulating a meaningful question, identifying and evaluating relevant information, preparing data for analysis, interpreting results, and communicating findings. A reproducible project makes both the reasoning and the computations behind these activities accessible to others.

This guide presents an organizational framework for research-oriented Data Science projects using **R**, **Quarto**, and a version-controlled repository. Its principal concern is how to organize the documents and computational components of a project so that the research can be understood, evaluated, and reproduced. It is a reference guide rather than a prescribed sequence of course assignments.

1.1 Two complementary dimensions of reproducibility

Computational reproducibility is the ability to obtain the same computational results by executing the documented procedures with the same inputs and a sufficiently equivalent software environment. It depends on the availability of data, code, dependencies, configuration, and execution instructions. When randomness is involved, seeds and other relevant settings must also be controlled; some computations may still vary across platforms or software versions.

Methodological reproducibility, as used in this guide, refers to making the research design and analytical decisions sufficiently explicit that another researcher can understand, scrutinize, and apply the same procedures. This includes the formulation of research questions, the choice of observations and variables, the treatment of missing data, the selection of statistical methods, and the interpretation of results. This term is used here as a practical organizing distinction; terminology varies across research fields.

These dimensions are complementary, but neither guarantees the other. A script may reproduce a numerical result without explaining why the calculation was appropriate. Conversely, a carefully justified report may be impossible to reproduce computationally if its code or data are unavailable.

Dimension	Guiding question	Principal evidence
Computational reproducibility	Can the calculations be executed again?	Data, executable code, dependencies, configuration, instructions

Dimension	Guiding question	Principal evidence
Methodological reproducibility	Can the research decisions be understood and evaluated?	Questions, assumptions, decision criteria, justifications, limitations

Reproducibility is distinct from **replication**, which commonly refers to investigating whether a result can be obtained again using new data or an independently conducted study. Usage of these terms differs across disciplines.

1.2 Relationship to CRISP-DM

The Cross-Industry Standard Process for Data Mining (CRISP-DM) provides a useful reference for organizing Data Science work: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. A research project, however, need not follow these phases literally.

In an academic investigation, researchers may formulate their own questions rather than translate an organizational client's objectives into analytical tasks. Available datasets may constrain the questions that can be answered, and exploration may motivate revisions to the original research design. Accordingly, the approach developed here is **CRISP-DM inspired**, rather than a strict implementation of CRISP-DM.

The methodology describes the research process; the repository architecture describes how to document and implement that process. They need not have identical structures. In particular, the organization of computational notebooks should reflect coherent analytical responsibilities, not necessarily the sequence of reporting deadlines.

2 Organization of a Reproducible Research Project

2.1 The repository as the unit of reproducibility

A single notebook is rarely sufficient to reproduce an entire investigation. Reproduction may also require input datasets, reusable code, package versions, configuration files, and instructions. The **repository** is therefore the natural unit for organizing the project's reproducible materials.

The repository should make it possible to determine:

1. Which inputs were used, and where they came from.
2. Which procedures generated each analytical result.
3. Which outputs were produced, and where they are located.

4. Which assumptions and decisions informed the analysis.
5. How another person can execute the relevant computations.

Version control, ordinarily with Git, records the evolution of source files and supports collaboration. A repository alone does not guarantee reproducibility: external data, software environments, file paths, and execution order still require attention.

2.2 A minimal directory structure

A small project can begin without separate directories for reusable functions or automation scripts:

```
project/
   README.md
   .gitignore
   renv.lock
   renv/
       activate.R
   data/
       raw/
       processed/
   notebooks/
   quarto/
       reports/
   docs/
        reports/
```

This example assumes a Quarto project rooted at `project/`, with `project.output-dir: docs`; Quarto generally preserves source-relative subdirectories within the output directory. The exact output location should be checked for the selected formats and Quarto version.

Location	Responsibility
`README.md`	Explain the project, prerequisites, and reproduction procedure
`_quarto.yml`	Configure Quarto project rendering
`data/raw/`	Hold original input data, unchanged where redistribution is permitted
`data/processed/`	Holds the processed data, that will be used in the research project
`notebooks/`	Hold executable analytical documents
`quarto/reports/`	Hold report source documents

Location	Responsibility
`docs/`	Hold generated, distributable outputs

A later project may add `src/`, `scripts/`, or a dependency lockfile. These should be introduced when they solve an actual organizational problem, not merely to imitate a large software project.

2.3 Source files and generated artifacts

Source files are the materials edited by researchers: `.qmd` documents, R code, configuration, and documentation. **Generated artifacts** are outputs created from those sources: rendered PDF or HTML reports, figures, tables, and prepared datasets.

Separating sources from generated outputs reduces ambiguity. A PDF report belongs in a publication or distribution location; the `.qmd` document that generates it belongs with the project's editable source materials. When practical, generated outputs should be reproducible from the committed sources rather than manually modified after rendering.

Not every generated file must be tracked in Git. The decision depends on whether the file is a required deliverable, how easily it can be regenerated, its size, and applicable data-sharing restrictions. Large or sensitive raw datasets may require an access procedure rather than inclusion in the repository.

2.4 Naming and paths

Use descriptive, stable filenames such as `data_quality.qmd` or `missingness_profile.qmd`. Numeric prefixes may help when a meaningful sequence exists, but should not imply that notebooks require a shared interactive session.

Prefer project-relative paths over machine-specific absolute paths. A project should not depend on a particular student's home directory or on files manually placed outside the documented repository structure. In R, tools such as `here` can help construct paths from a consistent project root, provided the root is configured and understood.

3 Computational Notebooks

3.1 The role of Quarto notebooks

A Quarto `.qmd` document combines prose, executable R code, and computational outputs. In this guide, a **notebook** is an executable analytical document whose main responsibility is to

perform and explain a coherent computational task.

A notebook is more than a collection of R commands. It should state what it is trying to establish, make its inputs explicit, document its procedure, present its outputs, and explain what those outputs imply for the investigation.

3.2 One analytical responsibility per notebook

A useful principle, inspired by the Unix philosophy, is to **do one thing and do it well**. Applied to notebooks, this means that each document should have a recognizable analytical purpose. A notebook might examine the structure of a dataset, assess data quality, or construct a prepared dataset. Each task may involve several functions and calculations.

The appropriate unit of modularity is a **coherent analytical task**, not an individual R statement. Creating a separate notebook for every minor calculation would introduce unnecessary fragmentation.

A notebook may be reused or revisited during multiple reporting periods. Conversely, a single report may draw on several notebooks. Thus notebook boundaries should not be dictated solely by reporting deadlines.

3.3 A standard notebook anatomy

A consistent internal structure makes notebooks easier to review and reproduce:

Element	Question addressed
Objective	What does this notebook seek to establish or produce?
Inputs	Which data, parameters, and dependencies are required?
Procedure	Which operations are performed, and in what order?
Outputs	Which tables, figures, datasets, or results are produced?
Interpretation	What do the results mean, and what limitations remain?

This is a conceptual template, not a requirement to use exactly five headings in every document. The explanation should be proportional to the complexity and significance of the task.

3.4 Independent execution and explicit dependencies

A fundamental rule is that **each notebook should run in a fresh R session**, without depending on objects left in memory by earlier interactive work. It may depend on files produced by another step, but such dependencies must be documented and reproducible.

For example, a notebook should load its own input dataset rather than assume an object called `data` already exists in the global environment. If a processed dataset is required, the project must explain how that dataset is created.

This principle helps prevent a common notebook failure: a document that appears to work interactively but fails when executed from beginning to end. A useful validation practice is to render the notebook in a clean session and inspect the resulting outputs and errors.

3.5 Computational transparency and reproducibility

Computational transparency means that the reader can understand which computations were performed. **Computational reproducibility** additionally requires that those computations can actually be executed again under documented conditions.

Readable code and explanatory text support transparency. Explicit dependencies, accessible inputs, controlled randomness, and clean execution support reproducibility. Neither should be treated as a substitute for the other.

4 Scientific Reports

4.1 Reports as scientific communication

The principal responsibility of a report is to communicate the investigation to a reader who may not inspect the code. It should explain **what** was investigated, **why** the question matters, **what** was found, and **how the findings should be interpreted**.

The report should also describe the analytical methods at the level necessary for scientific evaluation. The distinction between reports and notebooks is not that reports omit methodology: rather, reports explain and justify methods, while notebooks expose their computational implementation and intermediate results.

For example, a report may explain why observations with missing values were excluded and discuss the resulting limitations. A supporting notebook can show exactly how missingness was quantified, which rows were excluded, and how the resulting dataset was checked.

4.2 Reports and notebooks as complementary records

Concern	Report	Notebook
Research question	State and motivate it	Translate it into computational tasks where relevant
Methodological choices	Justify consequential choices	Implement and document them
Results	Present selected findings	Show detailed calculations and intermediate outputs
Interpretation	Explain implications and limitations	Interpret task-specific diagnostic results
Reproduction	Point to supporting materials	Provide executable evidence

The division is one of **emphasis and audience**, not an absolute separation of subject matter. Significant methodological decisions should appear in the report even when the corresponding operations are fully documented in notebooks.

4.3 Relationship to reporting milestones

Reports may be organized around deadlines, project reviews, or course stages. Computational notebooks need not follow that same schedule. One report can synthesize several notebooks, and one notebook can support more than one report.

To preserve traceability, reports should identify the relevant notebook or reproducible source for major computational findings. Where feasible, report tables and figures should be generated from the same underlying data and computations used in the notebooks, rather than manually transcribed.

5 Modular and Automated Workflows

5.1 Analytical versus software modularity

Two forms of modularity should be distinguished:

- **Analytical modularity** divides the investigation into coherent tasks, each represented by a notebook or another well-defined analytical component.
- **Software modularity** separates reusable computational logic into functions and modules that can be called from multiple places.

Analytical modularity can be learned and practiced before software modularity. A small project may reasonably contain most of its code directly in notebooks. Repeated or complex computations can later be extracted into functions.

5.2 Reusable R functions and `src/`

As an analysis grows, repeated calculations or transformations may be moved into R functions stored in `src/`. The notebook then focuses on the analytical task and its interpretation, while the function encapsulates a reusable operation.

For example, a simple missingness calculation could be defined as:

```
proportion_missing <- function(x) {
  mean(is.na(x))
}

proportion_missing(c(2, NA, 4, NA))
```

```
[1] 0.5
```

This example illustrates separation of responsibility, not a requirement that every one-line operation become a function. Extracting code is worthwhile when it improves clarity, consistency, testing, or reuse.

5.3 Executable scripts and `scripts/`

A `scripts/` directory can hold programs that coordinate tasks: acquiring data, generating derived datasets, validating outputs, or rendering a sequence of documents. In this convention, `src/` contains reusable computational building blocks, while `scripts/` contains entry points that execute a workflow.

This distinction is helpful but not universal. Some projects use different naming conventions or adopt a workflow manager. The important requirement is that each component has a clear responsibility and documented inputs and outputs.

5.4 From interactive analysis to an automated pipeline

A project may develop progressively:

Level	Main components	Main capability
Reproducible analysis	Data, notebooks, reports	Re-execute documented analyses
Modular computation	Add reusable R functions	Reduce duplication and isolate computational logic
Automated workflow	Add orchestration scripts or workflow tools	Run linked operations systematically

These levels represent increasing technical sophistication, not mandatory stages. A small research project may be fully adequate at the first level.

An automated workflow should make dependencies explicit: which inputs are required, which process produces each output, and what needs to be rerun after an input changes. For complex projects, workflow tools can automate dependency tracking; for smaller projects, clear instructions and a few well-designed scripts may suffice.

5.5 Reproducibility checks

Before considering an analysis reproducible, verify that:

1. The original inputs can be accessed and identified.
2. The project explains how to install or restore the required software environment.
3. Notebooks run from a clean session in a documented order, or independently where intended.
4. Dependencies between notebooks, scripts, and generated datasets are explicit.
5. Important outputs can be regenerated without undocumented manual edits.
6. Reports agree with the computational outputs that support them.
7. Methodological decisions and limitations are recorded in an appropriate location.

These checks connect the computational and methodological dimensions of reproducibility. A successful render is necessary evidence for computational reproducibility, but it does not by itself establish that the analysis was methodologically appropriate.

6 Concluding Principles

A reproducible Data Science project benefits from a separation of responsibilities:

- **Reports** communicate the research problem, justify methodological decisions, and interpret findings.
- **Notebooks** perform and explain coherent analytical tasks through executable documentation.

- **Reusable functions** implement computational operations that benefit from being shared.
- **Scripts and workflow tools** coordinate repeatable execution when the project requires automation.
- **The repository** connects these materials, their inputs, and their outputs into an understandable and reproducible whole.

The objective is not to impose a complex software architecture on every investigation. It is to make the research understandable and the computations reproducible, introducing additional structure only when it improves clarity, reliability, or maintainability.